План курса:

07.09 - вводная лекция, разбиение на бригады, ЛР 1 (настройка программ) - на дом
14.09 - проверка выполнения ЛР 1, запуск модели робота в симуляторе Gazebo
21.09 - ЛР 2 (сенсоры и актуаторы робота). Телеуправление роботом Jetbot, получение данных с камеры и лидара робота. Завести сенсоры и актуаторы в Gazebo - на дом
28.09 - проверка запуска сенсоров и актуаторов в Gazebo. Выдача задания 1: написание простейшей ноды движения на цель (aruco-маркер) – домашка на две недели
05.10 - PID-регуляторы
~~12.10~~ 19.10 - сдача задания 1.
~~19.10~~ 26.10 - одометрия, фильтрация и комплексирование данных. ЛР 3 (запуск одометрии на роботе)
~~26.10~~ 02.11 - SLAM 1. Выдача задания 2: восстановить траекторию робота по rosbag с данными сенсоров. ЛР 4: построение карты на роботе
~~02.11~~ 09.11 - SLAM 2. ЛР 5: локализация в построенной карте на роботе
~~09.11~~ 16.11 - сдача задания 2
~~16.11~~ 23.11 - Навигационный стек ROS Nav2. ЛР 6: запуск навигации робота
~~23.11~~ 30.11 - Манипуляторы и управление ими. Выдача задания 3: мобильная манипуляция в симуляторе. ЛР 7: управление манипулятором в симуляторе
~~30.11 - доделка лабораторных работ~~
07.12 - сдача задания 3
14.12 - зачет

Лабораторная работа 1. Настройка программ на ноутбуке с ОС Ubuntu

Краткое содержание:

- Установка и запуск ROS на ноутбуке.
- Импорт модели робота в симулятор Gazebo
- Установка и настройка Visual Studio Code

Дополнительные материалы:

- ROS1 и ROS2: https://yadi.sk/i/XhsQztEtrmJXYg
- Интеграция нейросетевых модулей в ROS: https://youtu.be/-Q4iLFDTpSE

Описание используемого оборудования и программного обеспечения:

Данная лабораторная работа является вводной и выполняется в рамках домашнего задания на ноутбуках. Эти ноутбуки в дальнейшем будут использоваться на лабораторных работах в том числе для удаленной работы с учебными роботами JetBot. Один робот выдается на учебную бригаду (2-3 человека). Студенты группы должны самостоятельно разделиться на бригады так, чтобы в каждой бригаде был хотя бы один ноутбук с операционной системой Linux (желательно - Ubuntu 20.04+). Если на ноутбуке стоит другая ОС, желательно поставить Ubuntu как вторую систему.

Лабораторная работа 2. Знакомство с роботом реальным и виртуальным. Работа с интерфейсами коммуникации, сенсорами, актуаторами и устройствами вывода через ROS.

Виртуальный робот в симуляторе Gazebo

1. **Установка ROS2 Humble + Gazebo на ноутбук:**

если не установлен ROS2 нативно, проще всего его установить в докере:

```
docker pull osrf/ros:humble-desktop-full
```

Далее нужно создать ROS Workspace (папка ros2_ws, в ней в папке src будут все наши ROS2-пакеты)

2. **Запуск Gazebo**

В ROS2 Gazebo запускается командой `ign`:

```
ign gazebo -v 4 -r visualize_lidar.sdf
```

Откроется окошко с газебовским миром и моделькой робота из шариков и параллелепипедов.

3. **Запуск Gazebo с моделькой джетбота на складе**

Сначала просто запустим Gazebo:

```
ign gazebo -v 4 -r
```

Если долго не запускается – можно закрыть и запустить еще раз, вполне возможно что запустится. Далее в открывшемся окошке выбираем “Tugbot in warehouse” – загрузится среда с моделью склада и моделью робота Tugbot на складе.

Вместо робота Tugbot хотелось бы видеть модельку нашего Jetbot-a.

Она есть в репозитории. Репозиторий нужно склонировать и собрать в воркспейсе как ROS2 пакет. Моделька робота имеет лидар и камеру, но топики с них сразу не видны в ROS - нужен ros_gz_bridge. Свяжем ROS и Gazebo согласно туториалу.

Затем нужно открыть файл `/root/.ignition/fuel/fuel.gazebosim.org/openrobotics/worlds/tugbot\ in\ warehouse/2/tugbot_warehouse.sdf` (например, с помощью расширения Docker-a для vscode) и вместо модельки Tugbot-a поставить туда ссылку на модельку Jetbot-a.

После встраивания модельки джетбота запустим Gazebo со средой Tugbot in warehouse:

```
ign gazebo -v 4 -r /root/.ignition/fuel/fuel.gazebosim.org/openrobotics/worlds/tugbot\ in\ warehouse/2/tugbot_warehouse.sdf
```

Реальный Jetbot

0. Подключение к роботу с ноутбука

Есть два способа подключения:

1. Через USB-интерфейс (кабель для подключения есть в коробке от робота). ip-адрес робота при подключении по USB 192.168.55.1.
2. Через Wi-Fi, если робот и компьютер подключены к одной сети. Если робот подключен к какой-либо сети, ip-адрес робота отображается на маленьком дисплее в задней части робота.

В обоих случаях, узнав ip-адрес робота, подключаемся к нему из терминала в ноутбуке через ssh:

```
ssh jetbot@<ip_address>
```

Пароль от всех роботов: `jetbot`

Для подключения робота к WiFi-сети через терминал можно воспользоваться утилитой nmtui:

```
$ sudo nmtui
```

Затем нужно выбрать Activate a Connection и выбрать сеть и ввести ключ доступа.

У роботов есть проблемы с датой, установим правильную дату и время вручную на роботе вот такой командой, чтобы дальше работала установка пакетов в докере:

`sudo date --set="2025-09-15 13:42:00"` (поменять на актуальные дату и время)

```
cd jetbot_ros2_mipt
```

Чтобы роботы не конфликтовали друг с другом, в файле docker-compose.yml устанавливаем ROS_DOMAIN_ID, равный номеру робота.

Далее запускаем контейнер с ROS2 и всеми необходимыми драйверами (образ уже есть на роботе собранный):

```
docker compose run -it jetbot_educational bash
```

1. **Запуск драйверов робота**

В контейнере чистим старую сборку и собираем заново workspace:

```
rm -rf build install log
```

```
colcon build
```

Затем устанавилваем переменные окружения командой source:

```
source install/local_setup.bash
```

Далее запускаем лаунчфайл для драйверов всех сенсоров робота и моторов:
`ros2 launch jetbot_bringup activate_all_drivers.launch.py`
Теперь, подключившись к роботу в другом терминале (с помощью команды `docker exec` …), можно увидеть топики лидара, камеры и моторов:

```
ros2 topic list
```

2. **Визуализация сенсоров робота в RViz на ноутбуке**

В запущенном на ноутбуке контейнере устанавливаем переменные окружения для дисплея и для ROS2 и запускаем RViz (команда `rviz2`)

```
source /opt/ros/humble/setup.bash
export DISPLAY=:1 (или export DISPLAY=:0, если не сработало)
export ROS_DOMAIN_ID=<jetbot_id>
rviz2
```

3. **Управление роботом с клавиатуры ноутбука**

Подключаемся к контейнеру, запущенному на ноутбуке (через docker exec), и там доустанавливаем пакет teleop_twist_keyboard и запускаем его:

```
apt-get update
apt-get install ros-humble-teleop-twist-keyboard
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

4. **Установка драйвера для камеры**

На джетботе стоит Pi Camera, поэтому нужно будет установить драйвер для нее (в контейнере):

```
cd src
git clone https://github.com/rpapallas/pi_camera_ros
cd ..
colcon build
```

Проверим, что она работает:

```
ros2 run pi_camera publish_compressed
```

Драйвер камеры публикует только сжатые изображения, чтобы публиковать несжатые, модифицируем код publish_compressed.py, создав новый файл publish_raw.py.

Добавим строчку для создания ROS-узла по скрипту publish_raw.py в файл setup.py:

```
entry_points={
    'console_scripts': [
     'publish_compressed = pi_camera.publish_compressed:main',
      ‘publish_raw = pi_camera.publish_raw:main’,
    'view_compressed = pi_camera.view_compressed:main',
    ],
},
```

И пересоберем `ros2_ws` командой `colcon build`

Рабочий пайплайн GStreamer для publish_raw:

```
pipeline = [
                                            'gst-launch-1.0', '-q',
                  'nvarguscamerasrc', f'sensor-id={self.sensor_id}',
                '!', (f'video/x-raw(memory:NVMM),width={self.width},'
```

```
f'height={self.height},format=NV12,framerate={self.fps}/1'),
                                                    '!', 'nvvidconv',
                                            '!', 'video/x-raw,format=BGRx',
                                                      '!', 'videoconvert',
                                             '!', 'video/x-raw,format=BGR',
                                  '!', 'filesink', 'location=/dev/stdout',
                                                                          ]
```

Порядок запуска ROS2-драйвера камеры в Docker-контейнере

1. Проверить работу камеры на хосте, вне контейнера

```bash
gst-launch-1.0 -q \
  nvarguscamerasrc num-buffers=1 sensor-id=0 \
                                                                        !
'video/x-raw(memory:NVMM),width=1280,height=720,format=NV12,framerat
e=30/1' \
  ! nvvidconv \
  ! 'video/x-raw,format=BGRx' \
  ! fakesink
```

Код возврата должен быть `0`:

```bash
echo $?
```

2. Пересоздать контейнер jetbot_educational, если он уже есть на роботе. Обычного `restart` недостаточно, потому что новые mounts, environment и группы применяются только при создании:

```bash
cd ~/IntelligentRoboticsOld/jetbot_ros2_mipt
docker compose config -q
docker compose up -d --force-recreate
```

****Команды ниже выполняются внутри контейнера****

3. Проверить NVIDIA-плагины внутри контейнера:

```bash
docker exec jetbot_ros2_mipt-jetbot_educational-1 bash -lc '
id
test -S /tmp/argus_socket
gst-inspect-1.0 nvarguscamerasrc >/dev/null
gst-inspect-1.0 nvvidconv >/dev/null
echo CAMERA_RUNTIME_OK
'
```

В выводе `id` должна быть группа `video`, а в конце — `CAMERA_RUNTIME_OK`.

4. Пересобрать workspace (достаточно только пакет камеры):

```bash
docker exec jetbot_ros2_mipt-jetbot_educational-1 bash -lc '
source /opt/ros/humble/install/setup.bash
cd /home/app/ros2_ws
colcon build \
  --symlink-install \
  --packages-select pi_camera \
  --event-handlers console_direct+
'
```

5. Запустить драйвер:

```bash
docker exec -it jetbot_ros2_mipt-jetbot_educational-1 bash
source /opt/ros/humble/install/setup.bash
source /home/app/ros2_ws/install/setup.bash
ros2 run pi_camera publish_raw
```

6. В другом терминале проверить публикацию:

```bash
docker exec -it jetbot_ros2_mipt-jetbot_educational-1 bash -lc '
source /opt/ros/humble/install/setup.bash
source /home/app/ros2_ws/install/setup.bash
```

```
ros2 topic hz /camera/image/raw
'
```

Ожидается примерно 28–30 FPS.

Калибровка камеры джетбота

Для калибровки камеры есть стандартный ROS2-пакет `camera_calibration`. Для его установки в терминале внутри контейнера введите:

```
sudo apt install ros-humble-camera-calibration
```

Перед запуском наклейте калибровочную доску на ровную твердую поверхность, посчитайте кол-во клеток в ней и одну сторону клетки (на распечатанной доске это должно быть 24 мм)

Далее запустите ноду калибровки:

```
ros2 run camera_calibration cameracalibrator \
    --size 7x10 \
    --square 0.024 \
    --ros-args \
    -r image:=/camera/color/image_raw \
    -p camera:=/camera
```

Откроется окошко калибровки, нужно будет водить доской перед камерой, показывая ее под разными углами, пока не наберется достаточно данных для калибровки. Далее нода сохранит результаты калибровки (матрицу камеры) в файл.

Лабораторная работа 3. Запуск и комплексирование одометрии на роботе и в Gazebo

1. **Трансформы робота**

Подключитесь к роботу и выведите дерево трансформов. Т.к. на роботе нет дисплея, придется визуализировать дерево через pdf:

```
ros2 run tf2_tools view_frames
```

Чтобы не перекидывать pdf с робота на ноутбук, можно визуализировать через окошко rqt на подключенном к роботу ноутбуке:
`ros2 run rqt_tf_tree rqt_tf_tree,`
выставив тот же ROS_DOMAIN_ID, что на роботе.
Откройте RViz на подключенном к роботу ноутбуке и выведите трансформы в нем (Add -> TF). Посмотрите на взаимное расположение фреймов на роботе.

2. Визуализация колесной одометрии

На роботе в activate_all_drivers.launch.py уже есть запуск колесной одометрии. Запустите драйвера, найдите в списке топиков топик одометрии, и визуализируйте его в RViz на ноутбуке. Установите джетбота на пол на точку старта в проходе между столами, отметьте эту точку (например, полоской скотча на полу) и перезапустите драйвера, чтобы сбросить одометрию в ноль. Сделайте круг вокруг стола на телеуправлении, вернувшись в ту же точку. Посмотрите на отклонение позиции из одометрии от нуля – в RViz и через `ros2 topic echo`. Сделайте еще два круга вокруг стола с возвращением в ту же точку и посмотрите еще раз на отклонение позиции из одометрии от нуля. Сделайте выводы о точности одометрии.

3. Лидарная одометрия

Для лидарной одометрии нужно установить пакет `rf2o_laser_odometry`. Склонируйте его в папку src вашего воркспейса на джетботе и соберите воркспейс. В лаунчфайле выключите публикацию одометрического трансформа, чтобы не конфликтовать с трансформом колесной одометрии (`publish_tf = false`). Запустите лаунчфайл, переключите топик одометрии в RViz на топик от rf2o, и покатайте робота вокруг стола, как в шаге 2. Сравните точность лидарной и колесной одометрии.

4. Комплексирование одометрии

Оба источника одометрии (колесная и лидарная) имеют свою ошибку. Хочется сделать из них одну общую одометрию, чтобы ее ошибка была меньше, чем у каждого из источников. Это можно сделать с помощью пакета `robot_localization`. Конфиг комплексирования задается в `params/ekf.yaml`. Основные настраиваемые параметры для каждого источника одометрии:

- odom0 – топик
- odom0_config – что именно берем в фильтр Калмана из 15 компонент состояния: координаты, углы Эйлера, скорости линейные, скорости угловые, ускорения линейные
- odom0_differential – нужно ли дифференцировать показания для получения скоростей

- odom0_relative – координаты этой одометрии относительные (например, колесная одометрия) или абсолютные (например, локализация по карте)

У колесной одометрии можно взять линейные и угловые скорости и углы ориентации. У лидарной одометрии – кооординаты и угол ориентации.

Нужно в папку params добавить новый конфиг на основе ekf.yaml, который будет соответствовать одометриям джетбота, и создать новый лаунчфайл в пакете robot_localization, который будет запускать фильтрацию с этим конфигом. Запустите обе одометрии (колесную и лидарную), и фильтрацию. Выведите все три одометрии в RViz разными цветами. Запомните точку старта робота и сделайте круг по комнате с возвращением в эту точку. Оцените отклонение каждой одометрии от нуля.